
Learning Shooting and Goalkeeping Skills for the Unitree G1 Humanoid

Team 12
CS2810 Embodied AI, Spring 2026

Abstract

We develop reinforcement-learning soccer skills for the 29-DoF Unitree G1 humanoid in the mjlabs MuJoCo simulator, covering the project’s two tasks: *shooting* (approach and kick a stationary ball into the goal) and *goalkeeping* (intercept a ball launched on a parabolic trajectory). For the shooter we build a six-stage progressive training pipeline around a shared recurrent policy, reaching a **99.96%** mean Phase-1 success rate (Phase-1 score $\approx 29.9/30$) while maintaining high shot speed and directional consistency. For tournament play, we deploy the shooter behind a multi-strategy API server whose strongest mode selects motions and shot targets online according to the goalkeeper’s observed position. For the goalkeeper, we organize the controller around an explicit two-phase design: a *prepare* phase that stabilizes the robot into a ready-to-dive stance before the shot is fully committed, and an *active* phase that executes the actual interception once the incoming trajectory is clear. On top of this structure, we use a staged pipeline that distills a diving prior, repairs difficult saves, and specializes interception behavior by ball region. The resulting Phase-1 goalkeeper reaches a **94.67%** block rate over the official 150-trial three-seed evaluation (Phase-1 score 22.0/30). For both tasks, we also describe how the competition server reconstructs policy observations directly from raw MuJoCo state so that deployment matches training-time observation construction.

1 Shooter

1.1 Phase 1 Results

Evaluation setup. We use the official Phase-1 shooting evaluation and supplement it with repeated large-scale evaluation over three fixed seeds to reduce variance and confirm stability.

Table 1: Shooter Phase-1 quantitative results.

Metric	Value
Seed 42 success rate	99.96%
Seed 2810 success rate	99.95%
Seed 202606 success rate	99.97%
Mean success rate	99.96%
Phase-1 score	29.9/30
Mean kick speed	13.03 m/s
Kick-direction cosine similarity	0.9787 ± 0.0191

Analysis. The Phase-1 shooter is already close to saturation: misses are extremely rare, and the small gap from full marks is due mainly to the grading rule magnifying the tiny gap from 100%. The main remaining weakness is far-corner precision under the hardest targets, not basic kick generation.

1.2 Phase 2 Design & Approach

Phase-2 approach. For Phase 2, we refine the strong Phase-1 single-agent shooter for tournament play through competition-frame alignment, motion selection, goalkeeper-aware target planning, and API-side deployment logic. The objective is to improve robustness against a reacting goalkeeper while preserving the high-accuracy, high-speed kicking behavior established in Phase 1.

1.2.1 Overview

Task. The G1 humanoid (29 DoF) performs a penalty-spot shot in MuJoCo: approach a stationary ball and kick it into the goal. This is a perception-guided motor skill requiring a motion-tracking prior and ball-trajectory control. Table 2 summarizes the deliverables.

Table 2: Shooter deliverables.

Artifact	Description
Progressive training pipeline	A six-stage curriculum that starts from motion imitation and gradually shifts toward competition-style ball control and scoring.
Final shooter policy	A recurrent policy that preserves motion consistency while reacting to the ball and shot destination online.
Competition policy layer	A multi-strategy API server that supports random shooting, best-motion selection, and goalkeeper-aware target planning.
Evaluation analysis	Motion-level accuracy and speed analysis used to identify which kicks remain reliable under different target regions.

1.2.2 Training pipeline: six-stage progressive training

Following the progressive perception–action framework of PAiD [1], we design six training stages. All stages share the same recurrent policy structure; what changes across stages is the task emphasis. The first stage establishes a stable motion prior by imitating kick trajectories. The next stage adds ball interaction, so the agent learns not just to reproduce motion clips but to make controlled contact with the ball. The middle stages then shift the objective toward scoring quality, explicitly favoring shots that are both accurate and fast enough to remain threatening in a competitive setting. Finally, the last stages align training fully with the competition world frame, so that the learned behavior transfers directly to the tournament setup rather than relying on an implicit train-to-deploy coordinate conversion.

Design conclusion. The main lesson from this curriculum is that high-quality kicking is easier to obtain by *re-weighting the objective across stages* than by searching for one perfect reward from the start. The early motion prior stabilizes learning; the later competition-aligned stages remove the remaining domain gap and produce the final near-saturated Phase-I performance.

1.2.3 Model architecture and observation space

Architecture and observations. The final shooter uses a recurrent actor–critic architecture because successful kicking is fundamentally sequential: approach, planting, swing timing, and shot redirection must remain consistent over multiple control steps. On the actor side, the observation is organized around three complementary sources of information: motion-reference state to preserve mechanically plausible kicks, proprioceptive state to stabilize the robot online, and task state describing the ball and desired shot destination in the robot’s local frame. During training, the critic is given additional privileged information for value estimation, but deployment relies only on the actor-side observation reconstructed from raw simulator state.

Design conclusion. Three conclusions follow from this design. First, recurrent state is essential: when the shot target is updated online, a memoryless controller is much more likely to break the kick sequence. Second, the observation should remain compact but structured, since mixing motion, proprioception, and task context is enough to support strong performance without needing a large hand-engineered state description. Third, exact deployment consistency matters more than aggressive robustness tricks at the final stage: once training is aligned to the competition frame, reducing observation and environment randomization improves transfer because it removes the dominant source of mismatch between training and tournament execution.

1.2.4 Reward design evolution

To make the curriculum progression explicit, Table 3 summarizes how the training objective shifts across stages.

Table 3: Core reward-term evolution across stages.

Reward term	S1	S2	S3	S4	S5	S6	Function
track_body_pos	1.0	1.0	0.5	0.5	0.05	0.05	body pose tracking
track_body_ori	1.0	1.0	0.5	0.5	0.05	0.05	body orientation tracking
track_anchor_ori	1.0	1.0	0.5	0.5	0.05	0.05	anchor orientation tracking
track_body_lin_vel	1.0	1.0	0.5	0.5	0.05	0.05	body linear-velocity tracking
track_body_ang_vel	1.0	1.0	0.5	0.5	0.05	0.05	body angular-velocity tracking
track_foot_pos	—	1.0	0.5	0.5	0.05	0.05	foot position tracking
tracking total	6.0	5.0	3.0	3.0	0.3	0.3	
contact	—	50	50	50	50	50	foot–ball contact force
sideways_kick	—	50	50	50	50	50	lateral kick toward goal
ball_vel_align	—	30	30	30	30	20	ball velocity aligned to goal
ball_speed	—	10	15	20	20	40	ball horizontal speed
goal_accuracy	—	—	10	10	30	20	goal-plane crossing accuracy
goal_miss	—	—	−5	−20	−50	−50	miss penalty
z_speed	—	0	0	0	−2	−5	vertical-speed penalty
ball_oob	—	—	—	—	−30	−30	out-of-bounds penalty

The reward design follows the same principle as the curriculum itself: early training emphasizes *stability and motion quality*, while later training shifts toward *shot outcome*. At the beginning, imitation-style tracking is necessary because the humanoid first needs a coherent kicking posture and timing. Once that prior is established, the reward increasingly favors meaningful ball contact, goal-directed velocity, and eventual scoring outcome. By the final stages, the optimization target is no longer “look like a kick” but “produce a fast, accurate, competition-valid shot.”

Design conclusion. The most important lesson is that successful humanoid shooting requires a *controlled handoff* from motion fidelity to task utility. If tracking remains too dominant, the robot reproduces kicks that look plausible but are not competitive. If task rewards dominate too early, the robot loses motion quality and becomes unstable. The staged schedule solves this by shifting the balance gradually rather than abruptly.

1.2.5 Tournament strategy and API server

API server. The shooter is deployed behind the standard competition API. A key design choice is that the server reconstructs the full policy observation from raw simulator state rather than relying on any shared feature interface. This keeps the learned policy compatible with the tournament protocol while preserving the exact observation structure used during training.

Observation computation. The server combines proprioception, ball state, motion-reference state, and the selected destination into the same observation layout used by the recurrent policy during training. This matters in practice because the shooter is sensitive not only to *what* target is chosen, but also to *when* that target enters the recurrent control loop.

Table 4: Shooting strategies.

Strategy	Trigger	Behavior	Latency
uniform	/reset	random motion (incl. left leg) + random target $y \in [-1.4, 1.4]$, locked immediately	none
best-motion	/reset	random target + <code>_select_motion_for_dest()</code> picks the best right-leg motion, locked immediately	none
gk-aware	per-frame /act	observe the GK for 5–16 frames, then lock the far-side target + best motion	5–16 frames

Three shooting strategies.

The gk-aware strategy. This is the default and most capable strategy, in three phases.

Observation phase (`shoot_planned["done"]==False`): each `/act` appends the GK root- y to `gk_y_history`; `_should_lock()` applies a dual condition – *behavioral trigger* (GK lateral offset $\geq$

0.3 m for a run of `min_observe= 5` frames $\rightarrow$ early lock) and *hard deadline* (reaching `deadline_pct= 60%` of the kick frame $\rightarrow$ force lock). Crucially we do **not** reset the policy on lock: locking only changes the `dest_body` observation term and leaves the LSTM hidden state intact, because the policy was trained on a continuous motion stream and jumping to frame 16 with an empty hidden state causes an out-of-distribution collapse.

Target planning (`_plan_destination`): if the GK is left ($y < -0.3$), shoot right `random(0.3, 1.4)`; if right ($y > +0.3$), shoot left `random(-1.4, -0.3)`; if central ($|y| \leq 0.3$), shoot the full goal width `random(-1.4, 1.4)`. The random component within the chosen half adds unpredictability.

Execution: `_select_motion_for_dest(dest_y, motions)` selects the best right-leg motion; the target world coordinate is set to $(-0.5, \text{dest}_y, 0.11)$ (goal-plane center); the motion continues to play and the policy steers the kick from the target term.

Target region $\rightarrow$ motion map. Built from `eval_motion_analysis.py` (5000 trials per motion $\times$ target):

Table 5: Target region to best motion (data-driven).

Target y range	Best motion	Speed	Accuracy	Reason
$[-1.5, -0.7)$ far left	006_right	16.2 m/s	0.10 m	best far-corner accuracy
$[-0.7, -0.3)$ mid-left	009_right	14.2 m/s	0.35 m	best right-leg mid-left accuracy
$[-0.3, +0.3]$ center	010_right	16.4 m/s	0.11 m	highest speed + good center accuracy
$[+0.3, +0.7]$ mid-right	005_right	12.1 m/s	0.006 m	best accuracy overall (sub-cm)
$[+0.7, +1.5]$ far right	005_right	15.7 m/s	0.64 m	most versatile motion

All selected motions are right-leg kicks. Left-leg motions are unused because they are slower (6–9 m/s) and completely fail the far-left corner (0% goal rate).

Kick-frame map and zero-GK baseline. `analyze_kick_timing.py` confirms all motions contact the ball at frames 26–30 (0.52–0.60 s) with zero std, which makes the `_should_lock()` deadline reliable (e.g. 001_right kicks at frame 27/260, 005_right at 26/230, 006_right at 27/225). When `compete.py` omits `-goalkeeper-api`, the GK is a `ZeroPolicy` (holds the `HOME_KEYFRAME` crouch at $y = 0$), giving a pure technical baseline.

1.2.6 Motion performance analysis

Scale: 10 motions $\times$ 5 targets $\times$ 5000 trials = **250,000** trials, model Stage 6 (`model_20000.pt`), targets $y = -1.50, -0.75, 0.00, +0.75, +1.50$ (even split of the 3.0 m goal).

Overall. Mean goal rate **94%** (9/10 motions at 100%); kick trigger rate 100%; right-leg mean ball speed 14.2 m/s (≈ 51 km/h); left-leg 8.0 m/s (≈ 29 km/h); right-leg mean accuracy 0.42 m; left-leg 0.26 m.

Table 6: Right-leg motion performance (all 100% goal rate).

Motion	Frames	Mean speed	Mean accuracy	Hit@0.3m	Best target	Best accuracy
001_right	260	13.59 m/s	0.340 m	40%	+0.75	0.152 m
004_right	257	14.02 m/s	0.548 m	20%	+0.75	0.197 m
005_right	230	13.96 m/s	0.402 m	20%	+0.75	0.006 m
006_right	225	14.56 m/s	0.343 m	40%	-1.50	0.095 m
009_right	205	13.73 m/s	0.454 m	40%	+0.00	0.144 m
010_right	230	15.40 m/s	0.512 m	20%	+0.00	0.112 m

Key findings. (1) *Strong right/left asymmetry:* right-leg ball speed is $\approx 2\times$ the left, and the left leg cannot reach the far-left corner – likely because right-leg motions are more numerous/better in the data or the reward structure favors the right leg. (2) *Far-corner accuracy degrades:* all motions lose accuracy at $y = \pm 1.50$ (0.4–1.2 m vs. 0.1–0.4 m at center), a reasonable mechanical limit. (3)

Table 7: Left-leg motion performance (80% goal rate; 0% only at $y = -1.50$).

Motion	Frames	Mean speed	Mean accuracy	Hit@0.3m
002_left	214	7.27 m/s	0.245 m	80%
003_left	289	7.88 m/s	0.277 m	65%
007_left	229	8.42 m/s	0.266 m	60%
008_left	225	8.36 m/s	0.261 m	60%

Table 8: Target $\times$ ball-speed matrix (m/s), right-leg motions.

Motion	$y=-1.50$	$y=-0.75$	$y=+0.00$	$y=+0.75$	$y=+1.50$
001_right	14.58	14.04	14.14	12.41	12.80
004_right	13.02	14.52	14.62	12.53	15.41
005_right	14.59	13.37	14.12	12.06	15.69
006_right	16.18	14.18	14.04	12.99	15.42
009_right	14.40	14.24	12.14	12.54	15.33
010_right	14.64	14.57	16.42	16.28	15.10

Competition recommendation: use right-leg motions only, with the `gk-aware` strategy that reads the GK during execution and shoots to the far side.

1.2.7 Engineering decisions and lessons

Unified architecture. All six stages share an identical LSTM (128-64-32 + 2×128), differing only by environment config. This lets checkpoints transfer seamlessly across stages (`-resume` loads actor+critic+normalizer), concentrates effort on reward/environment design rather than re-tuning networks, and speeds iteration (changing a reward weight needs no network redesign).

Coordinate alignment is a fatal detail. Stage 5 switching the training frame fully to `compete.py`’s world coordinates (robot (4, 0, 0.8), yaw π) looks trivial but is decisive: it removes the sim-to-sim domain gap. Checkpoints trained earlier in the training frame degrade markedly in the compete scene.

LSTM state continuity. Not calling `policy.reset()` on target lock in `gk-aware` was a key lesson. An initial implementation reset on `should_lock`, feeding the LSTM an empty hidden state at motion frame 16 – a combination never seen in training – causing an OOD collapse. The fix updates only the `dest_body` term and preserves LSTM continuity.

Data-driven motion selection. The `_select_motion_for_dest()` map is not heuristic but grounded in the 250K-trial measurements; each target region’s recommendation is backed by concrete speed/accuracy numbers, avoiding subjective guessing.

Rule-based vs. adversarial training. Phase 2 pits the shooter against an unknown opponent GK. Rather than *adversarial training* (which is unstable when two policies learn online and would compound an already complex six-stage curriculum), we chose a *strong single agent + rule-based target selection*: first train a near-perfect GK-free shooter (99.96% goal rate), then add a `gk-aware` rule that reads the GK and shoots to the far side. “Shoot to the far side” is a physically deterministic decision that needs no learning, and given a shooter that scores 12–16 m/s anywhere in the goal, this beats a random target by a wide margin against a reactive GK. The trade-off is reduced adaptivity to complex GK behavior (occlusion, tackles), but interpretability and controllability are preferable against an unknown opponent to a possibly under-trained adversarial policy.

2 Goalkeeper

2.1 Phase 1 Results

Evaluation setup. We use the official Phase-1 goalkeeping evaluation over three fixed seeds, with 50 trials per seed (150 total). A trial is counted as successful only if the ball does not enter the goal frame before timeout.

Table 9: Goalkeeper Phase-1 quantitative results.

Metric	Value
Seed 42 block rate	100.00%
Seed 2810 block rate	90.00%
Seed 202606 block rate	94.00%
Mean block rate	94.67%
Phase-1 score	22.0/30

Comparison. The evaluated goalkeeper remains clearly stronger than the single repaired network (81.7%) and the reference teacher from Ren et al. [2] ($\approx 72.5\%$ on our benchmark).

Analysis. The result is strong and stable across seeds, though high-corner saves remain the hardest cases. The spread from 90% to 100% across seeds suggests that most remaining misses are difficult boundary cases rather than systematic failures of the routing design.

2.2 Phase 2 Design & Approach

Phase-2 approach. For the tournament phase, we build on the strong Phase-1 goalkeeper and improve it through better repair, specialization, routing, and deployment logic, so that it transfers more reliably to the competition setting. The emphasis is on robustness and consistency under tournament-style play.

2.2.1 Task and the core difficulty

The goalkeeper must intercept a ball launched on a parabolic trajectory toward a full $3\text{ m} \times 1.8\text{ m}$ goal, controlling the G1 at 50 Hz. A save is credited only if the ball never crosses the goal plane within the frame. Crucially, the diving saves required to defend the corners necessarily terminate with the robot on the ground.

This single property reshapes the entire problem. Any learning signal that penalizes falling simultaneously penalizes the dives that produce saves, so the conventional “remain upright” inductive bias is actively counterproductive; the evaluation reflects this by not treating a fall as a failure. The consequence is that **the required interception behavior is not reachable by reward-shaped exploration** – under a hand-designed reward, a humanoid does not stumble into a committed, directional dive through random search. We verified this empirically: trained from scratch with PPO [3] under a range of shaped rewards, the policy reliably learns to stand and to satisfy the proxy terms, but its block rate saturates between 3% and 33%, with the high-ball corners frozen near zero. A weak reward yields passive standing ($\approx 33\%$, barely above the do-nothing baseline); a strong one induces an uncontrolled lunge that topples without intercepting.

The central design question is therefore not “which reward yields a goalkeeper” but rather **how to inject a diving prior, and how to subsequently improve upon it against the true objective**. The remainder of the method follows from this premise.

A second observation establishes the target. A per-region diagnosis of the Humanoid Goalkeeper of Ren et al. [2] – whose checkpoint serves as our teacher – shows that it blocks only $\approx 72.5\%$ on our benchmark, with failures concentrated almost entirely in the high balls (low balls $\approx 95\%$, the upper corners $\approx 40\text{--}60\%$). This is doubly informative: competent behavior still leaves the corners largely undefended, and it implies that **pure imitation of Ren et al. [2] cannot exceed $\approx 72.5\%$** . To do better, we must *synthesize* high-ball saving behavior that the teacher does not itself possess.

2.2.2 Method: a staged pipeline culminating in a region mixture-of-experts

Prepare and active phases. Our goalkeeper uses an explicit **two-phase control design**. The first is the *prepare* phase before the shot becomes fully threatening. Its purpose is not to make an early save, but to put the robot into a stable, ready-to-dive configuration: balanced over the support polygon, crouched to a useful height, and aligned laterally so that either-side launch remains available. This stage solves a practical failure mode we repeatedly observed in single-phase designs: if the policy begins reacting aggressively from the first moment, it often spends its control authority on

unnecessary sway, loses balance before the ball arrives, or enters the interception too upright to generate a decisive dive.

The second is the *active* phase after the ball trajectory is clearly committed toward the goal. Here the objective changes completely: the goalkeeper should stop preserving a neutral stance and instead execute the region-appropriate interception as quickly and decisively as possible. In other words, the design separates *stability preparation* from *save execution*. This separation proved important because the two phases favor different behaviors: before launch we want calm posture regulation, while after launch we want explosive lateral and vertical motion even if the save ends on the ground. Structuring the policy this way makes the keeper noticeably more stable at shot onset and gives the later diving controller a much better state from which to attack the ball.

Because the diving prior cannot be acquired from scratch and imitation of Ren et al. [2] is upper-bounded at $\approx 72.5\%$, no single training procedure is sufficient. We adopt a **staged pipeline** in which each stage overcomes the failure mode of the preceding one: distil a diving base (Stage A); *prove* with an offline optimizer that the conceded balls are physically saveable and fold those saves into one network (Stages B–C); *specialize* a bounded corrective head per ball region (Stage D); and route the specialists with an analytic gate into the deployed mixture (Stage E).

Stage A — Distillation: injecting the diving prior. We behavior-clone the teacher policy of Ren et al. [2] into a trainable network of our own design, since the released teacher is inference-only under the current RL framework [8] and cannot itself be optimized. The procedure follows DAGger [4]: the student is rolled out, the teacher labels the visited states, and the aggregated dataset is cloned. This yields a competent, reactive *diving* base policy with a 960-D history-based input. It does not surpass the teacher – imitation cannot – scoring $\approx 74\%$ over 100 trials and $\approx 66.9\%$ on the harder vectorized benchmark used in the rest of this part. It nevertheless provides a trainable policy that already executes dives, the precondition for every subsequent stage.

Stage B — Repair oracle: proving the hard balls are saveable. To exceed the base policy we must save high balls that it concedes. We construct a **per-scenario trajectory optimizer**: for the frozen base policy and a fixed incoming ball, it searches – using the cross-entropy method, CEM [6, 5], a sampling-based optimizer – an open-loop *residual* action sequence added to the base policy’s actions. The search minimizes a dense cost combining the true concede outcome with the distance from the nearest blocking limb to the ball’s predicted plane-crossing point. This optimizer **demonstrates that the hard balls are physically saveable** at a per-scenario save rate far above the base policy, establishing that control *authority* rather than kinematic reach was the limiting factor. The optimizer is single-agent simulation optimization, not a learned policy – it is used as a *data generator*, not as the deliverable.

Stage C — Distilling the repaired saves into a single network. The repaired residuals are per-scenario and open-loop; they cannot be deployed directly. We therefore **collect the repaired observation-action pairs from trajectories that the oracle turns into successful saves, and distil them back into a native MLP** with the same architecture as the base. The resulting single repaired network reaches **81.7%** – a +14.8-point gain over the 66.9% base. This single reactive network is strong but still pays the single-policy trade-off (improving one corner regresses another); it becomes the *shared base* that the region specialists of Stage D each refine. Two collection choices proved essential, each of which silently degraded the result when violated:

- *Demonstrations must be evaluation-matched.* Forcing a scenario after the environment reset leaves stale ball-history frames in the observation, training the policy on a distribution it never encounters at evaluation. The scenario must be imposed *through* the reset so that the observation history is consistent with the deployed keeper observation/history layout.
- *Collect a stable save window, not the post-save fall.* A naïve collection learns the high-ball save together with the subsequent topple-and-flail, so the deployed policy blocks but then loses balance after contact. The oracle residual is faded back to the base policy after the save window, and distillation keeps **only the frames around the keeper-plane crossing instant** (a short pre/post window), so the student learns the interception rather than the fall.

A **region-weighted, multi-GPU collection** path supports the same pipeline: one repair worker per GPU, with region weights that oversample low/mid balls (regions 4, 5 low, 0, 1 mid, 2, 3 upper) to

keep the distilled policy from overfitting to the dramatic high-ball jumps at the expense of routine balls.

Stage D — Per-region ballistic-residual specialists. A single reactive network cannot simultaneously hold six high-fidelity region dives – this is a capacity limit, not a tuning one. We therefore **freeze the repaired base and train one bounded residual head per ball region** (right and left $\times$ low, mid, up). Each head is zero-initialized – so training starts exactly at the strong base – and receives explicit *ballistic timing features*: the time and (y, z) of the ball at the keeper plane $x = 0$ and at the goal plane $x = -0.5$, the incoming v_x , and the ball speed. Trained with the stable save-window collection of Stage C and a region focus, each specialist masters its own balls without the cross-region trade-off that caps the single network. Because the residual is bounded and zero-initialized, a specialist can only *refine* the base dive for its region, never destabilize it.

Stage E — Mixture of experts with an analytic ballistic gate. At inference the six specialists are combined into a single deployable bundle and routed by an **analytic ballistic-crossing gate** – no learned parameters and no privileged information. From the raw ball position and velocity the gate extrapolates, in closed form, the ball’s state when it reaches the keeper plane: the crossing time t , the lateral crossing c_y , the crossing height c_z , and the descent rate v_z at crossing. Thresholds map c_z to a height band ($c_z < 0.85$ m $\rightarrow$ low, $c_z > 1.35$ m $\rightarrow$ up, else mid) and $c_y < 0$ to the left side, selecting one of the six experts. The region is **latched early** – as soon as the ball is unambiguously approaching ($v_x < -1$ and within 5 m of the keeper) – so the correct specialist drives the *entire* dive rather than a default expert mishandling the opening frames (which is fatal for fast balls to the far side). Finally, an optional **left/right symmetry mirror** serves a weak corner through its stronger mirrored expert, correcting the teacher’s residual left-bias. This mixture is what lifts the keeper above both the single repaired network and the $\approx 72.5\%$ teacher.

How the two-phase design fits the final system. The deployed keeper combines these ideas hierarchically. The *prepare* phase maintains a compact, balanced ready state while the ball is still in its pre-commitment window, ensuring that the robot enters the save from a controllable posture rather than from incidental motion. Once the incoming shot is sufficiently clear, the controller switches into the *active* phase and the analytic ballistic gate selects the region specialist that should own the dive. This division of labor is one of the main reasons the mixture works in practice: specialists are asked to solve the hard interception itself, not the separate problem of recovering from a poor pre-shot stance.

2.2.3 Additional design evidence and negative results

Three findings around the mixture are reported honestly: one clean negative that shaped the specialist training, one variance caution, and one in-progress path to push past the current best.

On-policy ballistic-residual PPO (negative). Before training the specialists from oracle data, we tried the obvious alternative: freeze the base and learn each bounded, zero-initialized residual head directly with PPO [3] on the block reward. The runs *stayed* near the frozen base and produced no clear gain – the conservative residual is safe but does not, on its own, synthesize the hard-ball saves. This negative is exactly why the Stage D specialists are trained by distilling the repair-oracle demonstrations rather than by on-policy reward: the productive signal lives in the oracle, not in on-policy exploration of the residual.

Gate-threshold sweeping (variance caution). The analytic gate has a few scalar thresholds (the height bounds and the latch distance), and short-run threshold sweeps can produce tempting candidates. We found these unreliable: a candidate that looked best on a few hundred trials regressed under a large-trial re-evaluation. We therefore fix the gate thresholds and trust only the 16,384-trial estimate, not short gate sweeps.

Failure mining and failure-replay PPO (in progress). To push past the current $\approx 93.8\%$, we also explored a pipeline that mines the mixture’s residual misses, replays those specific failing balls, and fine-tunes the responsible specialist on them. It is included as the next-stage path but is **not** the selected deployed policy, so we do not claim its gains here.

2.2.4 API server implementation and observation computation

For the cross-evaluation tournament the policy is served behind the standardized competition API. The tournament driver sends the **same raw MuJoCo state** to every team – root pose and velocity, joint positions and velocities, previous action, and ball position and velocity – and our server turns that raw state into exactly the observation tensor the policy was trained on. The match between deployment-time observation construction and training-time observation construction is what the entire mixture depends on, so we specify it concretely.

Per-frame observation (96-D), computed server-side from raw state. On each control step the server builds one 96-D frame by concatenating six terms, each transformed into the goalkeeper’s own pelvis frame with the exact scalings used in training (Table 10). Only the ball position and the keeper’s own proprioception are used; no privileged or opponent state enters the observation. The key non-obvious detail is that the reference frame and the per-term scalings must match training exactly – a wrong scale or a world-frame ball vector can silently destroy the policy at deployment even though the network itself loads without error.

Table 10: Goalkeeper per-frame observation (96-D) built from raw state.

Term	Dim	Computation from raw state
Ball position in pelvis frame	3	Relative ball position, rotated from world coordinates into the goalkeeper’s local frame
Base angular velocity	3	Root angular velocity rotated into the body frame and scaled consistently with training
Projected gravity	3	Gravity direction expressed in the body frame, giving an implicit orientation cue
Joint position offset	29	Current joint angles relative to the goalkeeper default pose
Scaled joint velocity	29	Joint velocities after the same normalization used during training
Previous action	29	The action applied at the previous control step

Term-major history stacking ($\rightarrow$ 960-D). The actor consumes a 10-frame history, but **not** as a naive frame-major concatenation. *Term-major* stacking is used: all 10 frames of ball position are placed first, then all 10 frames of base angular velocity, and so on per term, yielding the $96 \times 10 = 960$ -D vector the MLP expects. The server keeps a 10-slot history buffer; on the first frame after a reset it pre-fills all 10 slots with the initial frame so the very first action is well-defined.

Inference flow and mixture routing. During play, the server computes the 96-D frame, updates the history buffer, stacks it to 960 dimensions, and runs policy inference. Because the analytic gate needs the raw ball state, the server also tracks the ball position and velocity directly, extrapolates the ballistic crossing point, latches the region early in the episode, and lets the corresponding specialist control the rest of the dive. Reset clears the per-episode latch and history state so each new trial starts cleanly.

Deployment status. The main remaining integration requirement is to ensure that the tournament deployment path preserves this same history-based observation construction and mixture routing logic. A simplified single-frame server would not serve the goalkeeper correctly, so the deployed system must retain the same observation layout and routing behavior described above.

References

- [1] J. Kong, X. Liu, Y. Lin, J. Han, S. Schwertfeger, C. Bai, and X. Li, “Learning soccer skills for humanoid robots: A progressive perception-action framework,” 2026. arXiv:2602.05310.
- [2] J. Ren, J. Long, T. Huang, H. Wang, Z. Wang, F. Jia, W. Zhang, J. Wang, P. Luo, and J. Pang, “Humanoid goalkeeper: Learning from position-conditioned task-motion constraints,” arXiv:2510.18002, 2025.
- [3] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, “Proximal policy optimization algorithms,” arXiv:1707.06347, 2017.

- [4] S. Ross, G. J. Gordon, and J. A. Bagnell, “A reduction of imitation learning and structured prediction to no-regret online learning,” in *Proc. AISTATS*, 2011, pp. 627–635.
- [5] C. Pinneri, S. Sawant, S. Blaes, J. Achterhold, J. Stueckler, M. Rolinek, and G. Martius, “Sample-efficient cross-entropy method for real-time planning,” in *Proc. CoRL*, PMLR vol. 155, 2021, pp. 1049–1065.
- [6] R. Y. Rubinstein and D. P. Kroese, *The Cross-Entropy Method*. New York: Springer, 2004.
- [7] E. Todorov, T. Erez, and Y. Tassa, “MuJoCo: A physics engine for model-based control,” in *Proc. IEEE/RSJ IROS*, 2012, pp. 5026–5033.
- [8] N. Rudin, D. Hoeller, P. Reist, and M. Hutter, “Learning to walk in minutes using massively parallel deep reinforcement learning,” in *Proc. CoRL*, PMLR vol. 164, 2022, pp. 91–100.